DATASTRAW TECHNOLOGIES ASSESSMENT TEST

Hiring Assignment: Build an AI-Powered CX Reply Assistant

Submission Deadline: 3-4 days

Expected effort: ~10-12 hours

ABOUT DATASTRAW

Datastraw is building the operating infrastructure for modern businesses.

We believe the way businesses operate is changing fundamentally. The next generation of companies will rely on systems that bring together people, software, data, automation and intelligence - allowing teams to do more, move faster and scale without continuously adding operational complexity.

We're building that infrastructure, with customer experience as one of our core domains. Our technology brings together customer conversations, workflows, knowledge, integrations, data and AI to help consumer brands build and run scalable CX operations.

You'll be joining our core team at a stage where much of this is still being built. We already have an internal CRM, automation infrastructure and multiple integrations in place, but our technology function is still early. This role is an opportunity to shape the architecture, engineering practices and team that will underpin the next phase of Datastraw.

This assessment is designed to evaluate how you think, build, solve problems and take ownership - not simply how much code you can write.

PART 1 - BUILD

AI-Powered CX Reply Assistant

Build a small application that helps a customer support agent generate an AI-assisted response to a customer.

This should be a working, deployed application, not just a prototype on your local machine.

Scenario:

A CX agent is handling a customer conversation.

The customer says:

"My order was delivered but the bottle is broken. What can I do?"

The agent should be able to use your application to understand the conversation, access relevant brand information, and generate an appropriate response.

Core Requirements:

1. Conversation View

Create a simple interface showing:

- Customer name
- Brand
- Conversation history

- Latest customer message
- Basic order information

You may use mock data.

Additionally, include a way to switch between the Customer and Agent side of the conversation (e.g. a toggle or two views) so a message can be sent as the customer and the agent side can then generate/send a reply to it. This is meant to let you (and us, when reviewing) test the AI reply loop end-to-end, not just view seeded mock data**.**

2. Brand Knowledge Base

Create a knowledge base for at least two different brands, each containing at minimum:

- Return policy
- Refund policy
- Shipping policy
- Cancellation policy

The content can be mocked, but each brand's policies should differ meaningfully (e.g. different refund windows) so it's possible to verify that Brand A's knowledge is never used when responding to a Brand B customer.

Whatever retrieval approach you use (keyword matching, embeddings/vector search, or otherwise) is your choice for a KB this size, either is a defensible answer, and we're more interested in you being able to justify the trade-off than in which one you pick.

What matters is that the knowledge base content itself is manageable through the application, a create/edit/delete interface for KB entries per brand, even a simple one. This lets us change a policy ourselves and re-test the AI's behavior without touching your code or database directly.

The system should retrieve the relevant brand-specific information when generating a response.

3. AI Reply Generation:

The agent should be able to click:

Generate Reply

The application should:

1. Identify the relevant brand.
2. Retrieve relevant information from the knowledge base.
3. Provide the appropriate context to an LLM.
4. Generate a suggested customer response.
5. Display the response to the agent, along with the retrieved knowledge/context that was used to generate it.

The agent should then be able to:

- Edit the response
- Regenerate the response
- Approve the response
- Compose and send a manual message directly, independent of the AI, so the conversation supports a real back-and-forth, not just a single generate-and-approve action.

Actual WhatsApp or email delivery is not required; sending into the app's own conversation view is sufficient.

4. AI Guardrails:

The AI should not simply answer every question confidently.

For example, if the knowledge base says:

Refunds are only permitted within 7 days of delivery.

and the customer asks:

"I received this 20 days ago. Can I get a refund?"

The system should not confidently promise the customer a refund.

Think about how you would constrain the AI to the information available to it and what should happen when the required information is unavailable.

Also consider what happens when the knowledge base has no relevant information at all for the customer's question. What should the AI do in that case, and how should that be reflected to the agent?

5. Data & Logging:

Store the relevant information, such as:

- Customer message
- Retrieved context/knowledge
- AI-generated response
- Agent-edited response
- Final response
- Timestamp

You do not need to build a complete CRM.

We are looking for one well-built piece of a real CX product.

Technology:

You may use the stack you are most comfortable with.

However, our current technology environment includes:

- React
- Supabase
- PostgreSQL
- Supabase Auth
- Edge Functions / RPCs
- Qdrant
- OpenRouter
- DeepInfra
- REST APIs
- Webhooks
- Meta WhatsApp API

You do not need to use all of these.

We are more interested in your engineering decisions than in whether you have used every technology listed.

PART 2 - SYSTEM DESIGN

Imagine the application you have built is successful.
Datastraw now has:

- 500 brands
- 5,000 CX agents
- Millions of customer messages
- Multiple communication channels
- AI-generated responses
- Brand-specific knowledge bases

The system needs to reliably handle incoming customer messages, retrieve customer/order information, retrieve the correct brand knowledge, and generate AI-assisted responses.

Your Task:

Create one architecture diagram showing how you would design this system at scale.

Then explain your architecture in 2-3 pages maximum.

Please cover:

Architecture

How would you structure the:

- Frontend
- APIs/backend
- Database
- Authentication
- External integrations
- AI layer
- Knowledge retrieval
- Background jobs/queues, if required

Multi-Brand Data

How would you ensure:

Brand A's data, conversations, and knowledge can never accidentally be accessed by Brand B?

Explain where you would enforce this.

AI

How would you make the AI system reliable?

Consider:

- Knowledge retrieval
- Hallucinations
- Context provided to the model
- Confidence/fallback mechanisms
- Evaluation of AI responses

Scalability

If we went from 20 brands to 500 brands:

What do you think would break first, and what would you change?

Reliability

What would happen if:

- A webhook was received twice?
- An external API timed out?
- An AI request failed?
- A message was processed, but the response wasn't sent?

You don't need to build these systems.

We want to understand how you think about them.

PART 3 - TECHNICAL PROBLEM SOLVING

Answer the following scenario.

You do not need to write code unless you feel it helps explain your answer.

Scenario 1 - AI Costs

Datastraw's AI costs have increased from ₹20k/month to ₹1 lakh/month over three months.

The number of customers and conversations has only increased by approximately 40%.

What would you investigate?

And what would you potentially change?

Consider areas such as:

- Model selection
- Token usage
- Prompt size
- Retrieval
- Caching
- Unnecessary AI calls
- AI usage monitoring
- Rate limits
- Architecture

There is no single correct answer.

We are evaluating your problem-solving and decision-making process.

PART 4 - LEADERSHIP & OWNERSHIP

We are looking for someone who can eventually build and lead a technology team.

You don't need to have formally managed people before. If you haven't, tell us about situations where you have mentored, supported or taken ownership for someone less experienced.

Please answer these questions honestly and briefly.

1. Leadership

Have you previously led, mentored or managed someone more junior than you?

If yes:

- How many people?
- For how long?
- What were you responsible for?

- What was the most difficult part?

If not, tell us about a situation where you helped a less experienced teammate succeed.

2. Giving Feedback

A junior developer on your team repeatedly submits code that works but is poorly structured.

You've already discussed it with them once.

What would you do?

3. Disagreement

A developer on your team strongly disagrees with an architectural decision you have made.

How would you handle it?

4. Mistake & Ownership

Tell us about a technical mistake you have made that affected someone else or the business.

What happened, and what did you do after you realised it?

5. Joining Datastraw

You join Datastraw and discover that some of our existing technology processes are messy, documentation is incomplete, and there isn't always a clearly defined way of working.

What would your first 30 days look like?

AI USAGE

We actively encourage the use of AI tools.

You may use:

- ChatGPT
- Claude
- Cursor
- GitHub Copilot
- Gemini
- Stack Overflow
- Documentation
- Any other development tools

Please mention:

1. Which AI tools did you use?

2. How did you use them?

Give 2-3 examples.

3. What did AI get wrong?

Tell us about at least one instance where AI generated something that was incorrect, inefficient, or unsafe, and you had to identify or fix it.

We are not evaluating whether you used AI.

We are evaluating whether you can use AI effectively without outsourcing your thinking to it.

WHAT TO SUBMIT

After completing the assessment, please submit the following through the Google Form:

1. Deployed Application

A publicly accessible URL where we can test the application.

2. GitHub Repository

Please include:

- Source code
- README
- Setup instructions
- .env.example
- Database/schema information
- Clean folder structure

3. Architecture Diagram

A clear diagram of your proposed system.

4. Architecture Document

Maximum 2-3 pages.

5. Demo Video

Maximum 5 minutes.

Please cover:

- What you built
- How it works
- Your architecture
- Key technical decisions
- How you implemented the AI
- One thing you would improve with more time

6. Written Responses

Please provide your answers to Parts 3 and 4 in a Word document and upload it in the Google Form.

Note: All required links, documents, and the demo video must be submitted through the **Google Form:**
[Google Form Link - https://forms.gle/AdQUpTfUwJ2U3NVs8]

WHAT WE'RE LOOKING FOR

We're not looking for the most complex solution.

A strong submission will demonstrate that you can:

- Take an ambiguous problem and structure it.
- Build a working product end-to-end.
- Make sensible technical trade-offs.
- Think beyond the happy path.
- Understand how AI actually works within a product.
- Consider scalability, reliability, and security.
- Think about technology costs.
- Explain your technical decisions clearly.
- Take ownership when something goes wrong.
- Work well with people and eventually help others grow.

You will score higher for one well-designed, well-understood solution than for adding lots of unnecessary features.

IMPORTANT

You are not expected to know everything.

Someone with 2 years of experience may not have formally managed a team or designed systems at the scale described above.

We are interested in your potential to grow into the role, not whether you have already done everything.

During the next stage of the hiring process, we will discuss your submission with you and may ask you to explain your technical decisions.

Please make sure you understand the work you submit.

FINAL NOTE

This is not a traditional developer role.

You will be joining Datastraw's core team and will have the opportunity to help shape our technology function from an early stage.

We're looking for someone who is excited by the intersection of technology, AI and customer experience, enjoys solving ambiguous problems and wants to grow into a technology leadership role as Datastraw grows.

We'd much rather meet someone who is curious, takes ownership, and learns quickly than someone who knows every technology listed above but isn't excited about what we're building.

Please submit your completed assignment and all required files, links, and video through this **Google Form**